

# PrinterServer — Panoramica del Progetto

**Versione:** 1.2.1 — aggiornato al 07-05-2026

**Autore:** Roberto Ceccato — RC Projects

**Runtime:** Bun (non Node.js)

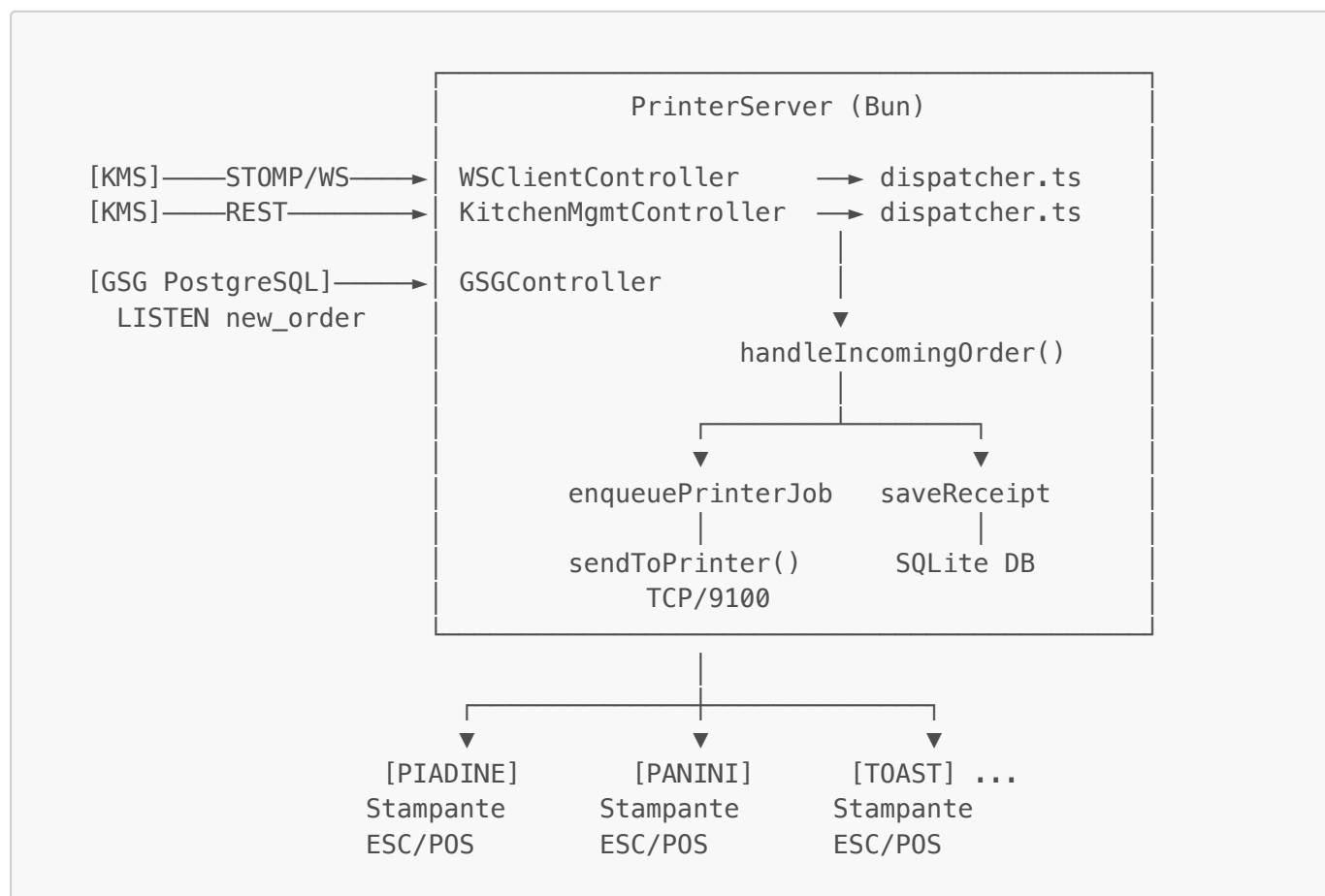
## 1. Scopo del progetto

PrinterServer è il backend che collega il sistema di gestione ordini della cucina (**KMS — Kitchen Management Server**) alle **stampanti termiche ESC/POS Ethernet** nei vari punti di produzione (PIADINE, PANINI, TOAST, PIATTI UNICI, COPERTI).

Riceve gli ordini in tempo reale via **WebSocket STOMP/SockJS** e in fase di sincronizzazione tramite **REST**, decide su quale stampante mandare il ticket, e lo stampa via **TCP sulla porta 9100**.

Espone anche un'interfaccia web (Angular SPA servita dalla cartella [www/](#)) per la gestione stampanti, la visualizzazione delle ricevute e le statistiche di cassa.

## 2. Architettura generale



## 3. File sorgente — [src/](#)

### 3.1 Entry point

## main.ts

Punto di ingresso dell'applicazione. Eseguito da Bun all'avvio.

Responsabilità:

- Legge e crea il file `.env` se assente (con valori di default)
- Inizializza nell'ordine:
  1. `DatabaseController` (SQLite)
  2. `HttpServerController` (server HTTP + WebSocket UI)
  3. `KitchenManagementController` (REST client verso KMS)
  4. `WSClientController` (STOMP/SockJS client verso KMS)
  5. `GSGController` (PostgreSQL LISTEN/NOTIFY)
  6. `seedPrintersIfDbEmpty()` + `loadPrintersFromDb()` (stampanti)
  7. `JobController` con 3 cron job attivi
- Gestisce `SIGINT/SIGTERM` per shutdown pulito

Variabili d'ambiente lette:

| Variabile                               | Default                                             | Descrizione                     |
|-----------------------------------------|-----------------------------------------------------|---------------------------------|
| DB_PATH                                 | <code>./data/db.sqlite</code>                       | Percorso file SQLite            |
| KITCHEN_MGMT_SERVER_URL                 | <code>http://127.0.0.1:8080</code>                  | URL del KMS                     |
| WS_CLIENT_RECONNECT_ATTEMPTS            | <code>-1</code> (infinito)                          | Tentativi riconnessione STOMP   |
| WS_CLIENT_RECONNECT_DELAY_MS            | <code>2000</code>                                   | Delay riconnessione STOMP       |
| TOKEN_KEY                               | <code>""</code>                                     | Secret per token auth           |
| HTTP_SERVER_PORT                        | <code>4000</code>                                   | Porta HTTP                      |
| GSG_DB_HOST/PORT/USER/PASSWORD/DATABASE | <code>localhost/5432/postgres/postgres/sagra</code> | PostgreSQL GSG                  |
| NODE_ENV                                | <code>development</code>                            | Ambiente                        |
| BASE_PATH                               | <code>""</code>                                     | Prefisso path per reverse proxy |

## 3.2 Logica di business

### dispatcher.ts

**Cuore dell'applicazione.** Riceve gli ordini normalizzati e li smista alle stampanti.

Funzioni esportate:

| Funzione                                    | Descrizione                                                           |
|---------------------------------------------|-----------------------------------------------------------------------|
| <code>setResyncCallback(fn)</code>          | Registra la callback di resync (evita circular dep.)                  |
| <code>setFetchItemCallback(fn)</code>       | Registra la callback per fetch singolo ordine per ID                  |
| <code>handleIncomingData(data)</code>       | Processa un messaggio STOMP grezzo del topic <code>/topic/pkmi</code> |
| <code>handleSingleOrderData(items[])</code> | Processa un array di ordini (da sync/resync REST)                     |
| <code>handleIncomingOrder(order)</code>     | Routing → idempotency → stampa su coda per-stampante                  |
| <code>reconcileMissedOrders(ids[])</code>   | Recovery ordini TODO→DONE persi nel debounce del resync               |
| <code>printSpecificOrder(id)</code>         | Ristampa un ordine specifico per ID (da API REST)                     |
| <code>printTestTicket(printerKey)</code>    | Stampa ticket di test su una stampante specifica                      |
| <code>regenerateSpecificReceipt(id)</code>  | Rigenera il buffer di un receipt esistente                            |

Costanti rilevanti:

- `RESYNC_DEBOUNCE_MS = 3000` — finestra di debounce per il resync
- `INTER_PRINT_DELAY_MS = 100` — pausa tra stampe consecutive sulla stessa stampante
- `_printerQueues: Map<string, Promise<void>>` — coda per-stampante (serializzazione)

Per il flusso dettagliato vedere [async-message-flow.md](#).

## print-routing.config.ts

Gestione della configurazione delle stampanti.

- `PrinterConfig` — tipo che descrive una stampante (key, name, ip, port, destination, active, upsideDown, beepEnable)
- `printers: PrinterConfig[]` — array globale caricato dal DB e usato ovunque
- `seedPrintersIfDbEmpty()` — popola la tabella `printers` se vuota (prima installazione)
- `loadPrintersFromDb()` — ricarica l'array `printers` dal DB (da chiamare dopo ogni modifica)

Destinazioni di default nel seed:

| Chiave                   | Destinazione |
|--------------------------|--------------|
| <code>piadine</code>     | PIADINE      |
| <code>panini</code>      | PANINI       |
| <code>toast</code>       | TOAST        |
| <code>piattiunici</code> | PIATTI UNICI |
| <code>coperti</code>     | COPERTI      |

## tickets.ts

Genera i buffer ESC/POS per la stampa.

| Funzione                                                                       | Descrizione                                                                           |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>buildKitchenTicket(order, dest, items)</code>                            | Builder v1 — usa <code>escpos-buffer</code> + logo PNG                                |
| <code>buildKitchenTicket_v2(order, dest, items, upsideDown, beepEnable)</code> | Builder v2 — usa <code>node-thermal-printer</code> , supporta stampa capovolta e beep |
| <code>buildSittingPlaceTicket(order, dest, items)</code>                       | Ticket per posto a sedere (non usato in produzione)                                   |
| <code>buildTestTicket(printerName)</code>                                      | Ticket di test per verifica connessione                                               |

Il builder v2 è quello attivamente usato. Stampa: logo MFO, destinazione, tipo piatto, flag asporto, numero ordine, tavolo, cliente, note piatto, note ordine.

## print.ts

Invio fisico del buffer alla stampante via TCP.

- `sendToPrinter(name, ip, port, buffer)` — wrapper pubblico; imposta flag `printing=true` per 5s
- `sendToPrinterInternal(...)` — connessione TCP con `Bun.connect`, timeout 1s, write buffer + `sock.end()`

## printer-status.ts

Polling dello stato hardware delle stampanti.

Usa comandi ESC/POS:

- `DLE EOT 1/2/3/4` — stato generale, offline, errori, carta
- `GS a 0` — comando alternativo per stampanti cinesi

Funzioni principali:

- `checkPrinterStatus(printer)` — interroga una stampante e aggiorna `printerStatusMap`
- `checkAllPrintersStatus(printers[])` — eseguito dal cron ogni 2 minuti
- `getPrinterStatus(name) / getAllPrinterStatuses()` — lettura stato corrente
- `setPrinterPrinting(name, value)` — flag in-stampa (settato da `print.ts`)

Invia `PrinterStatusChangeEvent` via WebSocket UI quando lo stato cambia.

## 3.3 Tipi e utilities

### types.ts

Definizioni TypeScript centrali.

| Tipo                                                  | Descrizione                                                         |
|-------------------------------------------------------|---------------------------------------------------------------------|
| <code>OrderItem</code>                                | Singolo articolo (nome, tavolo, cliente, note, dest, qty, takeAway) |
| <code>OrderPayload</code>                             | Ordine completo (orderId, orderNumber, status, items[])             |
| <code>ReceiptLog</code>                               | Ricevuta di stampa salvata nel DB                                   |
| <code>PrinterStatus</code>                            | Stato hardware stampante (online, paperEnd, coverOpen, ecc.)        |
| <code>PrinterStatusChangeEvent</code>                 | Evento di cambio stato                                              |
| <code>JobDefinition / JobInfo / UpdateJobInput</code> | Definizioni cron job                                                |

## logger.ts

Logger centralizzato con livelli:

- `debug(msg)` — solo se `DEBUG=true` o `NODE_ENV=development`
- `info(msg)` — sempre
- `warn(msg)` — sempre
- `error(msg)` — sempre

## utils.ts

- `groupBy<T>(array, getKey)` — raggruppa un array per chiave stringa (usato per raggruppare items per destinazione)

## users.ts

Gestione password con **Argon2id** ([@node-rs/argon2](#)):

- `hashPassword(plain)` — genera hash con salt (memoryCost 19MB, timeCost 2)
- `verifyPassword(plain, hashed)` — verifica

## gsg-helper.ts

Raccolta di query SQL PostgreSQL per il database del sistema cassa GSG.

Query disponibili:

- `totali` — KPI generali (ordini, incasso, scontrino medio, coperto/asporto)
- `perArea` — breakdown per area
- `trendOrario` — trend orario
- `perPagamento` — per tipo pagamento
- `piazzaVsAsporto` — dine-in vs asporto
- `topArticoli` — top 50 articoli venduti
- `topAggregati` — top categorie

- `righePerOrdineConTipologia / righePerOrdineConTipologiaUnprocessed` — righe ordini GSG

---

## 3.4 Controllers — `src/controllers/`

### `db.controller.ts` — `DatabaseController`

Singleton SQLite via `bun:sqlite` (API **sincrona** — nessun `await`).

Schema database:

| Tabella                       | Scopo                                                                     |
|-------------------------------|---------------------------------------------------------------------------|
| <code>receipts</code>         | Storico stampe — fonte idempotency ( <code>orderId</code> <b>UNIQUE</b> ) |
| <code>printers</code>         | Configurazione stampanti (ip, port, dest, active, ecc.)                   |
| <code>settings</code>         | Impostazioni generiche (inutilizzata al momento)                          |
| <code>barcodes</code>         | Scansioni barcode (code, timestamp, success)                              |
| <code>tokens</code>           | Token auth (non usati attivamente)                                        |
| <code>users</code>            | Utenti login (username, password Argon2id)                                |
| <code>gsg_orders</code>       | Ordini provenienti dal sistema GSG                                        |
| <code>todo_plate_cache</code> | Cache persistente plate originale ordini TODO                             |

Metodi principali:

| Metodo                                           | Descrizione                             |
|--------------------------------------------------|-----------------------------------------|
| <code>getReceiptByIdAndStatus(id, status)</code> | Check idempotency pre-stampa            |
| <code>saveReceipt(log)</code>                    | Salva ricevuta post-stampa              |
| <code>getPrinterSettings()</code>                | Lista stampanti                         |
| <code>savePrinterSettings(data)</code>           | Inserisce/aggiorna stampante            |
| <code>saveTodoPlate(orderId, plateName)</code>   | INSERT OR IGNORE plate originale        |
| <code>getTodoPlate(orderId)</code>               | Legge plate originale                   |
| <code>deleteTodoPlate(orderId)</code>            | Rimuove dopo stampa                     |
| <code>getAllTodoPlateCache()</code>              | Lista completa per reconciliation       |
| <code>backupDatabase(path)</code>                | Backup file SQLite (mantiene ultimi 10) |
| <code>checkIntegrity()</code>                    | PRAGMA integrity_check                  |
| <code>getUserByUsername(username)</code>         | Auth login                              |

---

### `httpserver.controller.ts` — `HttpServerController`

Singleton Elysia (framework HTTP per Bun). Gestisce tutte le route REST e il WebSocket push verso la UI.

**Autenticazione:** token base64 con payload `{exp, key}` — scadenza 10 ore. Verificato via header `Authorization: Bearer <token>`.

**Route REST:**

| Metodo | Path                                        | Descrizione                        |
|--------|---------------------------------------------|------------------------------------|
| POST   | <code>/api/login</code>                     | Autenticazione — restituisce token |
| POST   | <code>/api/verify-token</code>              | Verifica validità token            |
| GET    | <code>/assets/*</code>                      | File statici Angular (www/assets/) |
| GET    | <code>/media/*</code>                       | File media (immagini ecc.)         |
| WS     | <code>/api/ws</code>                        | WebSocket push notifiche UI        |
| GET    | <code>/api/ws/status</code>                 | Lista client WS connessi           |
| GET    | <code>/api/receipts</code>                  | Lista ricevute (con filtri)        |
| POST   | <code>/api/receipts/:id/print</code>        | Ristampa ricevuta per ID           |
| POST   | <code>/api/receipts/:id/printAt</code>      | Ristampa su stampante specifica    |
| GET    | <code>/api/printers/getAll</code>           | Lista stampanti configurate        |
| POST   | <code>/api/printers/add</code>              | Aggiunge stampante                 |
| POST   | <code>/api/printers/update</code>           | Aggiorna stampante                 |
| POST   | <code>/api/printers/delete/:key</code>      | Elimina stampante                  |
| POST   | <code>/api/printers/saveAll</code>          | Salva batch di stampanti           |
| POST   | <code>/api/printers/test/:key</code>        | Stampa ticket di test              |
| GET    | <code>/api/printers/status</code>           | Stato hardware tutte le stampanti  |
| GET    | <code>/api/printers/status/:name</code>     | Stato hardware stampante specifica |
| POST   | <code>/api/printers/check-status</code>     | Forza check stato immediato        |
| GET    | <code>/api/barcodes/getAll</code>           | Lista scansioni barcode            |
| POST   | <code>/api/barcodes/add/:id</code>          | Registra scansione barcode         |
| GET    | <code>/api/statistics/totals</code>         | KPI generali GSG                   |
| GET    | <code>/api/statistics/trend</code>          | Trend orario GSG                   |
| GET    | <code>/api/statistics/by-area</code>        | Per area GSG                       |
| GET    | <code>/api/statistics/by-payment</code>     | Per tipo pagamento GSG             |
| GET    | <code>/api/statistics/channel</code>        | Piazza vs asporto GSG              |
| GET    | <code>/api/statistics/top-products</code>   | Top prodotti GSG                   |
| GET    | <code>/api/statistics/top-categories</code> | Top categorie GSG                  |
| GET    | <code>/api/statistics/by-cashier</code>     | Per cassa GSG                      |

| Metodo | Path                                      | Descrizione        |
|--------|-------------------------------------------|--------------------|
| GET    | <code>/api/statistics/by-table</code>     | Per tavolo GSG     |
| GET    | <code>/api/statistics/departments</code>  | Per reparto GSG    |
| GET    | <code>/api/statistics/total-covers</code> | Totale coperti GSG |

**WebSocket (`/api/ws`) — tipi di notifica broadcast:**

| Tipo                               | Quando                                       |
|------------------------------------|----------------------------------------------|
| <code>NEW_TICKETS</code>           | Ogni ordine in ingresso (prima della stampa) |
| <code>RECEIPT_PRINTED</code>       | Stampa completata con successo               |
| <code>RECEIPT_PRINT_FAILED</code>  | Stampa fallita                               |
| <code>PRINTER_STATUS_CHANGE</code> | Cambio stato hardware stampante              |
| <code>ping</code>                  | Heartbeat periodico (cron PING — commentato) |

### `kitchenmgmt.controller.ts` — `WSClientController` + `KitchenManagementController`

**`WSClientController`** — Client STOMP/SockJS verso il KMS.

- Singleton con riconnessione automatica (backoff su retry)
- `_pkmiQueue`: `Promise<void>` — coda seriale messaggi STOMP
- All'`onConnect`: esegue sync iniziale REST + registra callback `resync` e `fetchItem` + sottoscrive topic
- Topic: `/topic/greetings` (log) e `/topic/pkmi` (ordini)

**`KitchenManagementController`** — Client REST verso il KMS.

- `fetchItems(statuses[], pageSize?)` — GET `/plate-item?statuses=...` con auto-paginazione via `Promise.all`
- `fetchItemById(id)` — GET `/plate-item/{id}` — usato da `reconcileMissedOrders`

### `gsg.controller.ts` — `GSGController`

Client PostgreSQL per il sistema cassa GSG. Usa `LISTEN/NOTIFY`.

- Si connette al DB e fa `LISTEN new_order`
- Heartbeat ogni N secondi per mantenere la connessione attiva
- Backoff esponenziale (1s → 15s) su errore/disconnect con riconnessione automatica
- Alla notifica `new_order` → `onNotification()` → `handleIncomingOrderFromGSG()` in dispatcher
- `queue[]` + `isProcessing` — coda interna per serializzare i notify in arrivo

### `cron.controller.ts` — `JobController`

Scheduler cron basato su `node-cron`. Singleton con `EventEmitter`.

API: `create(def)`, `get(id)`, `pause(id)`, `resume(id)`, `runNow(id)`, `remove(id)`, `stopAll()`



**Job attivi:**

| ID                   | Cron        | Descrizione                                           |
|----------------------|-------------|-------------------------------------------------------|
| GSG_QUEUE            | ogni 10 min | Processa coda ordini da GSG (predisposto, task vuota) |
| DB_BACKUP            | ogni 30 min | Backup file SQLite (mantiene ultimi 10)               |
| PRINTER_STATUS_CHECK | ogni 2 min  | <code>checkAllPrintersStatus(printers)</code>         |

**statistics.controller.ts — StatisticsController**

Wrapper che esegue le query da `gsg-helper.ts` sul client PostgreSQL GSG.

- `getTotals(startDate, endDate)` — KPI
- `getTrend(startDate, endDate)` — trend orario
- `getByArea(...)` — per area
- `getByPayment(...)` — per pagamento
- (e così via per tutte le query GSG)

**4. Dipendenze esterne principali**

| Libreria                                         | Scopo                                             |
|--------------------------------------------------|---------------------------------------------------|
| <code>elysia</code>                              | Framework HTTP per Bun                            |
| <code>stompjs + sockjs-client</code>             | Client WebSocket STOMP                            |
| <code>node-thermal-printer</code>                | Generazione buffer ESC/POS (builder v2)           |
| <code>escpos-buffer + escpos-buffer-image</code> | Generazione buffer ESC/POS (builder v1, logo PNG) |
| <code>node-cron</code>                           | Scheduler cron job                                |
| <code>pg</code>                                  | Client PostgreSQL per GSG                         |
| <code>bun:sqlite</code>                          | SQLite nativo Bun (sincrono)                      |
| <code>@node-rs/argon2</code>                     | Hashing password Argon2id                         |
| <code>dotenv</code>                              | Lettura file <code>.env</code>                    |

**5. Dati persistenti — SQLite (`data/db.sqlite`)**

```
data/
  db.sqlite                ← database attivo
  db.sqlite.backup-<timestamp>.sqlite ← backup automatici (ultimi 10)
  gsg_db/
    new_order_function.sql ← DDL funzione PostgreSQL GSG
```

**6. Frontend — `src/www/`**

Angular SPA pre-compilata (bundle di produzione). Servita direttamente da `HttpServerController` come file statici.

Funzionalità UI:

- Gestione stampanti (lista, aggiunta, modifica, test, stato hardware)
- Storico ricevute e ristampa
- Statistiche di cassa GSG
- Configurazione sistema

Localizzazione: `assets/i18n/en.json` e `it.json`.

## 7. Integrazione con sistemi esterni

Kitchen Management Server (KMS)

- **STOMP WebSocket** (`/ws` con SockJS) → topic `/topic/pkmi` — ordini in tempo reale
- **REST GET** `/plate-item?statuses=TODO,PROGRESS` — sync iniziale e resync
- **REST GET** `/plate-item/{id}` — fetch singolo ordine per reconciliation

GSG (sistema cassa)

- **PostgreSQL LISTEN** `new_order` — notify su nuovo ordine
- **REST queries** (via `StatisticsController`) — statistiche per la dashboard UI

Stampanti termiche ESC/POS

- **TCP port 9100** — invio buffer via `Bun.connect`
- **DLE EOT / GS a** — polling stato hardware (online, carta, errori)

## 8. Flow globale — avvio sistema

```

1. main.ts
  | DatabaseController.getInstance(dbPath)
  |   | initializeDatabase() → crea tutte le tabelle
  |
  | HttpServerController.instance
  |   | Elysia listen(:4000)
  |
  | KitchenManagementController.getInstance(url)
  |
  | WSClientController.getInstance(options)
  |   | connect() → SockJS → STOMP
  |   |   | onConnect:
  |   |   |   | fetchItems(TODO, PROGRESS) → handleSingleOrderData → stampa
  |   |   |   |
  |   |   |   | setResyncCallback(...)
  |   |   |   | setFetchItemCallback(...)
  |   |   |   | subscribe /topic/pkmi → _pkmiQueue → handleIncomingData
  |   |
  | PROGRESS
  |
  | GSGController.getInstance(pgConfig)
  |   | start() → connectAndListen() → LISTEN new_order
  
```

```
|  
├─ seedPrintersIfDbEmpty() + loadPrintersFromDb()  
├─ JobController.getInstance()  
│   ├── GSG_QUEUE (ogni 10min)  
│   ├── DB_BACKUP (ogni 30min)  
│   └── PRINTER_STATUS_CHECK (ogni 2min)
```

Per il dettaglio del flusso degli eventi STOMP e della logica di stampa, vedere [async-message-flow.md](#).